

QUERY PROCESSING LAB EXP 11 to 20

NAME : UDHAYAKUMAR U

REG NO : 192424332

COURSE CODE : DSA0503 - Query Processing

EXP 11

Aim

To create a DataFrame with 10 rows and 4 columns containing random values, convert some values to NaN, and highlight the NaN values using Pandas Styler.

Algorithm

- ❖ Import pandas and numpy libraries.
- ❖ Generate a DataFrame of random values (10×4).
- ❖ Replace selected values with NaN.
- ❖ Define a function to highlight NaN values.
- ❖ Apply the style function using `DataFrame.style.applymap()`.
- ❖ Display the styled DataFrame.

Code

```
import pandas as pd
import numpy as np

# Create DataFrame
df = pd.DataFrame(np.random.randn(10,4),
                  columns=['A','B','C','D'])

# Convert some values into NaN
df.iloc[2,1] = np.nan
df.iloc[5,3] = np.nan
df.iloc[8,0] = np.nan

# Function to highlight NaN values
def highlight_nan(x):
    if pd.isna(x):
        return 'background-color: yellow'
    return ""

print(df)

styled_df = df.style.applymap(highlight_nan)
styled_df
```

OUTPUT

```
>>> Type "help", "copyright", "credits" or "license()" for more information.
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qp.py =====
      A      B      C      D
0  2.502157  0.209561  0.323618 -0.272487
1 -0.439418  0.725284  0.154209  0.593740
2 -0.466533      NaN  1.117383  0.610740
3  0.155615 -0.040383  0.840178  0.948760
4 -1.464922  0.785373  1.224720 -0.711853
5  0.329356  0.059345  0.565486      NaN
6 -2.396203  0.227457 -0.753925  0.222411
7  0.732337 -0.870038  0.754400  0.584702
8      NaN  1.144976  0.721906 -0.161838
9 -0.244819  0.093158  0.484413  2.019682
Warning (from ipynb kernel):
```

EXP 12

Aim

To create a DataFrame with random values and set the DataFrame background color to black and font color to yellow.

Algorithm

- ❖ Import pandas and numpy.
- ❖ Create a DataFrame with random values.
- ❖ Use Pandas Styler.
- ❖ Set background color to black.
- ❖ Set font color to yellow.
- ❖ Display the styled DataFrame.

CODE

```
import pandas as pd

import numpy as np

df = pd.DataFrame(np.random.randn(10,4),
                  columns=['A','B','C','D'])

print(df)

styled_df = df.style.set_properties(**{
```

```
'background-color':'black',  
'color':'yellow'  
})
```

styled_df

OUTPUT

```
>>> ===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qg.py =====  
      A         B         C         D  
0 -0.512674 -0.369085 -0.241803  0.268291  
1  0.528922  0.299317  0.344387 -0.620593  
2  0.128897  1.768976 -0.491872 -0.042665  
3  0.761340 -0.236363 -0.727077 -1.096341  
4  0.308720  0.259685  2.687984  0.840829  
5 -0.605052 -1.076337 -0.384897  0.852002  
6  0.395753 -0.104957 -0.389361 -0.539265  
7  0.069902  0.256564  1.522820 -1.337393  
8 -2.012010 -0.421169  0.291643  0.964299  
9 -2.502390  0.739836 -1.042717  1.235368  
>>> |
```

EXP 13

Aim

To detect missing values in a DataFrame and display True or False using Pandas.

Algorithm

- ❖ Import pandas and numpy.
- ❖ Create a DataFrame.
- ❖ Insert NaN values.
- ❖ Use `isnull()` to identify missing values.
- ❖ Display the Boolean result.

CODE

```
import pandas as pd
import numpy as np

data = {
    'Name':['A','B','C','D'],
    'Age':[20,np.nan,22,21],
    'Marks':[90,85,np.nan,88]
}

df = pd.DataFrame(data)

print("Original DataFrame")
print(df)

print("\nMissing Values")
print(df.isnull())
```

OUTPUT

```
>>>
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qg.py =====
Original DataFrame
   Name  Age  Marks
0    A  20.0   90.0
1    B   NaN   85.0
2    C  22.0    NaN
3    D  21.0   88.0

Missing Values
   Name  Age  Marks
0  False False False
1  False  True False
2  False False  True
3  False False False
>>>
```

EXP 14

Aim

To find and replace the missing values in a DataFrame using Pandas.

Algorithm

- ❖ Import pandas and numpy.
- ❖ Create a DataFrame with NaN values.
- ❖ Use `fillna()` to replace missing values.
- ❖ Display the updated DataFrame.

CODE

```
import pandas as pd

import numpy as np

data = {
    'Name':['A','B','C','D'],
    'Age':[20,np.nan,22,np.nan],
    'Marks':[90,85,np.nan,88]
}

df = pd.DataFrame(data)
```



```
print("Original DataFrame")
```

```
print(df)
```

```
df = df.fillna(0)
```

```
print("\nDataFrame after replacing missing values")
```

```
print(df)
```

OUTPUT

```
>>>
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qp.py =====
Original DataFrame
   Name  Age  Marks
0    A  20.0   90.0
1    B   NaN   85.0
2    C  22.0    NaN
3    D   NaN   88.0

DataFrame after replacing missing values
   Name  Age  Marks
0    A  20.0   90.0
1    B   0.0   85.0
2    C  22.0    0.0
3    D   0.0   88.0
>>>
```

EXP 15

Aim

To keep only the rows that contain at least 2 NaN values in a DataFrame.

Algorithm

- ❖ Import pandas and numpy.
- ❖ Create a DataFrame containing NaN values.
- ❖ Count the number of NaN values in each row.
- ❖ Filter rows having two or more NaN values.
- ❖ Display the filtered DataFrame.

CODE

```
import pandas as pd
```

```
import numpy as np
```

```
data = {  
    'A':[1,np.nan,np.nan,4],  
    'B':[np.nan,np.nan,3,4],  
    'C':[1,2,np.nan,np.nan],
```

```
'D':[4,np.nan,np.nan,5]
}
```

```
df = pd.DataFrame(data)
```

```
print("Original DataFrame")
```

```
print(df)
```

```
result = df[df.isnull().sum(axis=1) >= 2]
```

```
print("\nRows with at least 2 NaN values")
```

```
print(result)
```

OUTPUT

```
>>> 3      D      0.0      88.0
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qp.py =====
Original DataFrame
   A      B      C      D
0  1.0  NaN  1.0  4.0
1  NaN  NaN  2.0  NaN
2  NaN  3.0  NaN  NaN
3  4.0  4.0  NaN  5.0

Rows with at least 2 NaN values
   A      B      C      D
1  NaN  NaN  2.0  NaN
2  NaN  3.0  NaN  NaN
>>>
```

EXP 16

Aim

To split a DataFrame into groups based on School Code and check the type of the GroupBy object.

Algorithm

- ❖ Import the Pandas library.
- ❖ Create a DataFrame containing student details.
- ❖ Group the DataFrame using the School_Code column.
- ❖ Display each group.
- ❖ Display the type of the GroupBy object.

CODE

```
import pandas as pd
```

```
data = {  
    'School_Code': ['S001','S002','S001','S002','S003'],  
    'School_Name': ['ABC','XYZ','ABC','XYZ','PQR'],  
    'Age': [15,16,14,15,16]  
}
```

```
df = pd.DataFrame(data)
```

```
group = df.groupby('School_Code')
```

```
print("Groups:")
```

```
for name, value in group:
```

```
    print("\nSchool Code:", name)
```

```
    print(value)
```

```
print("\nType of GroupBy Object:")
```

```
print(type(group))
```

OUTPUT

```
>>> ===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qp.py =====
Groups:

School Code: S001
  School_Code School_Name  Age
0         S001         ABC   15
2         S001         ABC   14

School Code: S002
  School_Code School_Name  Age
1         S002         XYZ   16
3         S002         XYZ   15

School Code: S003
  School_Code School_Name  Age
4         S003         PQR   16

Type of GroupBy Object:
<class 'pandas.core.groupby.generic.DataFrameGroupBy'>
>>> |
```

EXP 17

Aim

To split a DataFrame by School Code and find the Mean, Minimum, and Maximum Age. .

Algorithm

- ❖ Import Pandas.
- ❖ Create a DataFrame.
- ❖ Group data using School_Code.
- ❖ Calculate Mean, Minimum, and Maximum age.
- ❖ Display the result.

CODE

```
import pandas as pd
```

```
data = {  
    'School_Code':['S001','S001','S002','S002','S003'],  
    'Age':[15,14,16,15,17]  
}
```

```
df = pd.DataFrame(data)
```

```
result = df.groupby('School_Code')['Age'].agg(['mean','min','max'])
```

```
print(result)
```

OUTPUT

```
===== Restart: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qp.py =====  
      mean  min  max  
School_Code  
S001      14.5   14   15  
S002      15.5   15   16  
S003      17.0   17   17
```

EXP 18

Aim

To split a DataFrame into groups based on School Code and Class.

Algorithm

1. Import Pandas.
2. Create a DataFrame.
3. Group the DataFrame using School_Code and Class.
4. Display each group.

CODE

```
import pandas as pd
```

```
data = {  
    'School_Code':['S001','S001','S002','S002','S003'],  
    'Class':['V','VI','V','VI','V'],  
    'Student':['John','Mary','David','Alex','Sam']  
}
```

```
df = pd.DataFrame(data)
```

```
group = df.groupby(['School_Code','Class'])
```

```
for name, value in group:  
    print("\nGroup:", name)  
    print(value)
```


OUTPUT

```
>>>
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qp.py =====

Group: ('S001', 'V')
  School_Code Class Student
0      S001     V   John

Group: ('S001', 'VI')
  School_Code Class Student
1      S001    VI   Mary

Group: ('S002', 'V')
  School_Code Class Student
2      S002     V   David

Group: ('S002', 'VI')
  School_Code Class Student
3      S002    VI   Alex

Group: ('S003', 'V')
  School_Code Class Student
4      S003     V   Sam
>>>
```

EXP 19

Aim

To display the dimensions (shape) of the World Alcohol Consumption dataset and extract the column names.

Algorithm

1. Import Pandas.
2. Read the dataset using `read_csv()`.
3. Display the dataset shape.
4. Display the column names.

CODE

```
import pandas as pd

# Read the CSV file
df = pd.read_csv("world_alcohol.csv")

# Display first five records
print("World Alcohol Consumption Dataset")
print(df.head())

# Display dimensions
print("\nShape of Dataset:")
print(df.shape)

# Display number of rows and columns
rows, cols = df.shape
print("\nNumber of Rows :", rows)
print("Number of Columns :", cols)
```

```
# Display column names
print("\nColumn Names:")
print(df.columns.tolist())
```

OUTPUT

```
>>> FileNotFoundError: [Errno 2] No such file or directory: 'world_alcohol.csv'

===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qg.py =====
World Alcohol Consumption Dataset
  Year WHO region Country Beverage Types Display Value
0  1986      AFR  Algeria          Wine           0.5
1  1986      AFR  Angola          Beer           5.8
2  1985      AFR   Benin        Spirits           1.2
3  1986      AMR  Canada          Beer          89.2
4  1987      EUR   France          Wine          45.8

Shape of Dataset:
(5, 5)

Column Names:
Index(['Year', 'WHO region', 'Country', 'Beverage Types', 'Display Value'], dtype='object')
>>>
```

EXP 20

Aim

To find the index position of a given substring in a DataFrame column.

Algorithm

- ❖ Import Pandas.
- ❖ Create a DataFrame.
- ❖ Use `str.find()` to locate the substring.
- ❖ Display the index position.
- ❖

CODE

```
import pandas as pd
```

```
df = pd.DataFrame({  
    'Name': [  
        'Alexander',  
        'Catherine',  
        'Jonathan',  
        'Michael'  
    ]  
})
```

```
substring = "an"
```

```
df['Index_Position'] = df['Name'].str.find(substring)
```

```
print(df)
```

OUTPUT

```
>>>
= RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/qp.py
      Name  Index_Position
0  Alexander             4
1  Catherine            -1
2   Jonathan             6
3   Michael            -1
>>> |
```